TP FINAL

Ingeniería de Datos en Azure / Microsoft Fabric

Documento Técnico

Arquitectura · Decisiones de Diseño · Modelo de Datos · Manual de Ejecución

Plataforma:
Microsoft Fabric (OneLake · Data Factory · Spark Notebooks)

Workspace:
gt-deuler-dev

Año 2026

1. Introducción

Este documento describe el diseño, la implementación y el manual de operación del **Trabajo Práctico Final de Ingeniería de Datos** desarrollado íntegramente sobre **Microsoft Fabric**. El proyecto consiste en la construcción de un pipeline de datos de extremo a extremo que integra múltiples fuentes heterogéneas, aplica transformaciones progresivas y entrega un modelo dimensional listo para consumo analítico.

La plataforma elegida es Microsoft Fabric, un entorno SaaS unificado que combina almacenamiento en OneLake, procesamiento distribuido con Apache Spark, orquestación con Data Factory y análisis con Power BI, todo bajo una misma experiencia de usuario. Esta elección elimina la necesidad de gestionar infraestructura y permite concentrar el esfuerzo en la lógica de negocio y en las buenas prácticas de ingeniería de datos.

1.1 Objetivos del trabajo práctico

- Construir un pipeline de datos completo con arquitectura Medallion (Landing → Bronze → Silver → Gold).
- Integrar dos fuentes de datos distintas: archivos CSV transferidos por SFTP y una base de datos relacional (AdventureWorks).
- Implementar lógica de carga incremental y total con persistencia de watermarks en una tabla de configuración centralizada.
- Construir un modelo dimensional en la capa Gold que permita análisis de ventas por fecha, cliente y producto.
- Parametrizar completamente el pipeline desde tablas de configuración, sin hardcodear rutas, nombres ni fechas.
- Integrar la API pública de Feriados Nacionales de Argentina y la API de clima Open-Meteo como fuente de enriquecimiento de la dimensión calendario.

1.2 Alcance y fuentes de datos

El trabajo procesa datos del dominio de ventas de la empresa ficticia **Adventure Works**. Las fuentes involucradas son:

- Fuente SFTP (archivos CSV): tablas SalesOrderHeader y Person, con separador punto y coma, cargadas con convención de nombre SalesOrderHeader_AAAAMMDD.csv. SalesOrderHeader se ingiere de forma incremental por fecha de archivo; Person se carga en su totalidad en cada ejecución.
- Fuente base de datos AdventureWorks (ADVWDB): tablas SalesOrderDetail (incremental por ModifiedDate) y Product (carga total). Esta fuente omite la capa Bronze y escribe directamente en Silver, dado que los datos provienen de un sistema transaccional confiable con estructura conocida.

- API externa Feriados Argentina: endpoint público del Gobierno Argentino que devuelve los feriados nacionales por año, consumido en la construcción de la dimensión calendario.
- API externa Open-Meteo: servicio meteorológico gratuito que provee datos históricos de temperatura, integrado como bonus de enriquecimiento en la dimensión calendario.

2. Arquitectura del Sistema

2.1 Arquitectura Medallion

El sistema adopta la **arquitectura Medallion** de tres capas, un patrón ampliamente utilizado en plataformas de datos modernas (Databricks, Microsoft Fabric, Azure Synapse). Cada capa tiene una responsabilidad específica y un contrato de calidad de datos bien definido:

Capa	Lakehouse	Propósito	Garantía
Landing	lh_landing	Zona de aterrizaje transitoria para archivos crudos CSV	Archivo original sin modificar, trazabilidad completa
Bronze	lh_bronze	Datos ingeridos como string, formato Delta, sin transformación de tipos	Dato completo, sin pérdida. Idempotente.
Silver	lh_silver	Datos limpios, tipados, con validaciones. Listos para analítica	Dato confiable, tipado correctamente, sin duplicados
Gold	lh_gold	Modelo dimensional (star schema): dimensiones y tabla de hechos	Dato optimizado para consultas analíticas y BI

> *Adicionalmente se utiliza lh_config como lakehouse de configuración centralizada, que persiste el estado del sistema (watermarks, IDs de notebooks, rutas, PKs) y desacopla la lógica de negocio del código.*

2.2 Flujo de datos de extremo a extremo

El recorrido completo de un dato desde su origen hasta el modelo analítico sigue la siguiente secuencia:

1. Los archivos CSV son depositados por el sistema fuente en el lakehouse lh_origin_get_talent (workspace externo WS-GT-DEMOS), simulando la recepción por SFTP.
2. El pipeline DP_INGST_SilverSFTP consulta la tabla lh_config.dbo.origin_bronze_silver para obtener la lista de tablas activas con origen = 'SFTP'. Por cada tabla, ejecuta en secuencia: (a) Copy Activity que transfiere los archivos CSV a lh_landing, (b) Notebook Bronze que lee el CSV y escribe en lh_bronze con todos los campos como string, (c) Notebook Silver que lee Bronze, tipa, limpia y escribe en lh_silver.
3. El pipeline DP_INGST_SilverADVWDB procesa las tablas de AdventureWorks. Al no tener capa Bronze, el notebook lee directamente de la base de datos con un TOP 1000 (ambiente de desarrollo) y escribe en lh_silver mediante un proceso de staging que garantiza la atomicidad.

4. Una vez completadas ambas ingestas Silver, el pipeline DP_ORCHS_Gold itera sobre los notebooks Gold registrados en lh_config.dbo.origin_gold y los ejecuta en orden estricto: DimCalendario, DimCliente, DimProducto, FactVentas.
5. DP_ORCHS_Origenes actúa como orquestador maestro: lanza DP_INGST_SilverSFTP y DP_INGST_SilverADVWDB en secuencia y, al completarse exitosamente, invoca DP_ORCHS_Gold.

2.3 Decisión arquitectónica: ADVWDB omite capa Bronze

Una decisión de diseño relevante es que las tablas provenientes de AdventureWorks (SalesOrderDetail y Product) **no pasan por la capa Bronze**. Esta decisión se fundamenta en:

- La fuente es una base de datos relacional con esquema conocido y tipos de dato nativos, por lo que no existe la ambigüedad propia de un archivo CSV.
- La capa Bronze existe para preservar el dato crudo en caso de archivos mal formados o de parsing incorrecto. Para una fuente SQL, este riesgo no aplica.
- Esta decisión fue alineada explícitamente con la consigna del trabajo práctico, que indica que los notebooks ADVWDB operan directamente sobre Silver.

> *La escritura en Silver para ADVWDB utiliza un patrón de staging temporal: los datos se escriben en una tabla auxiliar (lh_silver.staging.*) y luego se realiza un MERGE o INSERT SELECT hacia la tabla destino, garantizando atomicidad y evitando estados inconsistentes.*

3. Decisiones de Diseño

3.1 Parametrización total desde tabla de configuración

Ningún pipeline ni notebook contiene **rutas, nombres de tablas, fechas ni identificadores hardcodeados**. Toda la configuración del sistema reside en **lh_config.dbo.origin_bronze_silver** y **lh_config.dbo.origin_gold**. Esta decisión tiene múltiples ventajas:

- Agregar una nueva tabla al pipeline requiere únicamente insertar una fila en la tabla de configuración, sin modificar ningún notebook ni pipeline existente.
- El estado del sistema (último valor de watermark procesado) persiste de forma durable entre ejecuciones, sobreviviendo reinicios o fallos parciales.
- La auditoría y el diagnóstico son simples: el estado completo del pipeline se puede consultar con un SELECT sobre lh_config.

La tabla origin_bronze_silver contiene los siguientes campos por tabla configurada:

Origen	Tabla	Tipo	Carga	Campo WM	WM Ini.	PKs
SFTP	SalesOrderHeader	sftp_csv	incremental	fecha_archivo	20260221	SalesOrderID
SFTP	Person	sftp_csv	total	—	—	BusinessEntityID
ADVWDB	SalesOrderDetail	sql_db	incremental	ModifiedDate	2014-01-01	SalesOrderID, DetailID
ADVWDB	Product	sql_db	total	—	—	ProductID

3.2 Obtención de IDs de lakehouses en notebooks

Un aspecto crítico en notebooks ejecutados dentro de pipelines de Fabric es la obtención del Workspace ID y los Artifact IDs de cada lakehouse. En entornos de ejecución de pipeline (Livy sessions), el contexto de runtime no está disponible a través de notebookutils.runtime.context. La solución correcta y robusta es:

```
_lh_silver  = notebookutils.lakehouse.get('lh_silver')
_lh_bronze  = notebookutils.lakehouse.get('lh_bronze')
WS_ID       = _lh_silver.workspaceId
ART_SILVER  = _lh_silver.id
```

Este patrón funciona tanto en ejecución manual del notebook como cuando es invocado por un pipeline. Los métodos notebookutils.runtime.context y notebookutils.lakehouse.list() fallan silenciosamente en contexto de pipeline, devolviendo None y diccionario vacío respectivamente, lo que genera errores 400 Bad Request al construir rutas ABFS con WS_ID = None.

3.3 Normalización de rutas ABFS

Los paths ABFS hacia Files/ en OneLake requieren el prefijo **'Files/'** de forma explícita. El valor ubicacion_relativa almacenado en config contiene rutas relativas (ej: TP_AdventureWorks/Sales), por lo que los notebooks aplican la siguiente normalización antes de construir la URL:

```
_path = path_relativo.strip('/')
if not _path.startswith('Files/'):
    _path = 'Files/' + _path
abfs = f'abfss://{WS_ID}@onelake.dfs.fabric.microsoft.com/{ART}/{_path}'
```

3.4 Gestión de capacidad Spark (error 430)

El entorno de prueba de Microsoft Fabric tiene límites de concurrencia de sesiones Spark. Cuando el pipeline ejecuta múltiples notebooks en secuencia sin tiempo suficiente entre ellos, el sistema responde con HTTP 430 TooManyRequestsForCapacity. La solución implementada incorpora dos actividades Wait dentro del ForEach del pipeline DP_INGST_SilverSFTP:

- Wait1 (90 segundos): entre el Copy Activity y el notebook Bronze, para que cualquier sesión de iteraciones previas libere recursos antes de iniciar la ingesta.
- Wait2 (180 segundos): entre el notebook Bronze y el notebook Silver, para garantizar que la sesión Spark del Bronze se cierre completamente antes de que arranque Silver.

> *Este parámetro es ajustable en el JSON del pipeline. En capacidades de producción con SKU mayor, los tiempos de espera pueden reducirse o eliminarse.*

3.5 Carga incremental y watermarks

Para las tablas con tipo_carga = 'incremental', el sistema mantiene el campo **ultimo_valor_incremental** en lh_config. El notebook lee este valor al inicio de la ejecución, filtra únicamente los registros nuevos y actualiza el watermark al finalizar exitosamente. Este patrón garantiza:

- Idempotencia: una ejecución interrumpida antes de actualizar el watermark puede repetirse sin duplicados.
- Trazabilidad: el historial de valores de watermark permite reconstruir qué datos fueron procesados en cada ejecución.
- Reproceso: para reprocesar un rango de datos, basta con actualizar manualmente el campo ultimo_valor_incremental en lh_config.

4. Componentes del Sistema

4.1 Inventario de artefactos

Tipo	Nombre	Función
Lakehouse	lh_landing	Zona de aterrizaje de archivos CSV originales
Lakehouse	lh_bronze	Datos ingeridos, todo tipo string, formato Delta
Lakehouse	lh_silver	Datos limpios y tipados, listos para analítica
Lakehouse	lh_gold	Modelo dimensional optimizado para consumo
Lakehouse	lh_config	Tablas de configuración maestras del sistema
Pipeline	DP_INGST_SilverSFTP	Orquesta Landing → Bronze → Silver (fuente SFTP)
Pipeline	DP_INGST_SilverADVWDB	Orquesta Silver directo (fuente AdventureWorks DB)
Pipeline	DP_ORCHS_Origenes	Orquestador maestro de todas las fuentes
Pipeline	DP_ORCHS_Gold	Orquesta ejecución secuencial de notebooks Gold
Notebook	NB_CNFGS_SetupLhConfig	Carga inicial de tablas de configuración
Notebook	NB_INGST_BronzeSFTP_*	Ingesta CSV → Bronze (SalesOrderHeader, Person)
Notebook	NB_TRNSF_SilverSFTP_*	Transformación Bronze → Silver SFTP
Notebook	NB_TRNSF_SilverADVWDB_*	Extracción ADVWDB → Silver (SalesOrderDetail, Product)
Notebook	NB_TRNSF_GoldVentas_*	Construcción de dimensiones y fact table Gold

4.2 Pipelines de orquestación

DP_ORCHS_IngestaOrigenes — Orquestador maestro

El pipeline **DP_ORCHS_IngestaOrigenes** constituye el **punto de entrada principal del sistema de procesamiento de datos**. Su objetivo es coordinar la ejecución de los pipelines responsables de la ingesta de datos desde las diferentes fuentes y la posterior generación del modelo analítico.

El flujo de ejecución se realiza de forma **secuencial**, siguiendo el siguiente orden:

1. **DP_INGST_SilverSFTP**
   Pipeline encargado de la ingesta de archivos CSV provenientes de un servidor **SFTP**. Los archivos son copiados hacia la capa Landing del Lakehouse y posteriormente procesados mediante notebooks que generan las capas Bronze y Silver.
2. **DP_INGST_SilverADVWDB**
   Pipeline encargado de la ingesta de datos provenientes de una **base de datos SQL Server (AdventureWorks)**. Los datos son extraídos mediante una actividad Copy y cargados en tablas de staging en la capa Silver, donde posteriormente se aplican las transformaciones necesarias.
3. **DP_ORCHS_Gold**
   Pipeline responsable de ejecutar los notebooks que generan el **modelo analítico final en la capa Gold**, utilizado para análisis y consumo por herramientas de Business Intelligence.

En caso de fallo en cualquiera de los pipelines ejecutados, el proceso finaliza mediante una **actividad Fail**, generando un mensaje de error descriptivo que facilita el monitoreo y diagnóstico de incidentes.

Este pipeline es invocado automáticamente mediante un **trigger programado**, configurado en la tabla de configuración:

lh_config.dbo.trigger_origins

.

DP_INGST_SilverSFTP — Ingesta SFTP

Implementa el patrón: Lookup → Filter → ForEach. El Lookup lee todas las filas activas de origin_bronze_silver; el Filter selecciona únicamente las de origen = 'SFTP'; el ForEach (secuencial) itera y, por cada tabla, ejecuta: Copy Activity (CSV a lh_landing), Wait1 (90s), Notebook Bronze, Wait2 (180s), Notebook Silver. Todos los valores de rutas, IDs de notebook y separadores se leen de la fila correspondiente en config, sin hardcodeos.

DP_INGST_SilverADVWDB — Ingesta base de datos

Misma estructura Lookup → Filter → ForEach, pero filtrando origen = 'ADVWDB'. Al no existir capa Bronze para esta fuente, en cada iteración el pipeline ejecuta primero una Copy Activity que extrae los datos desde SQL Server hacia una tabla staging en Silver y posteriormente ejecuta el Notebook Silver que realiza las transformaciones y la carga final, que internamente maneja la lógica de staging y MERGE/INSERT según el tipo de carga de la tabla.

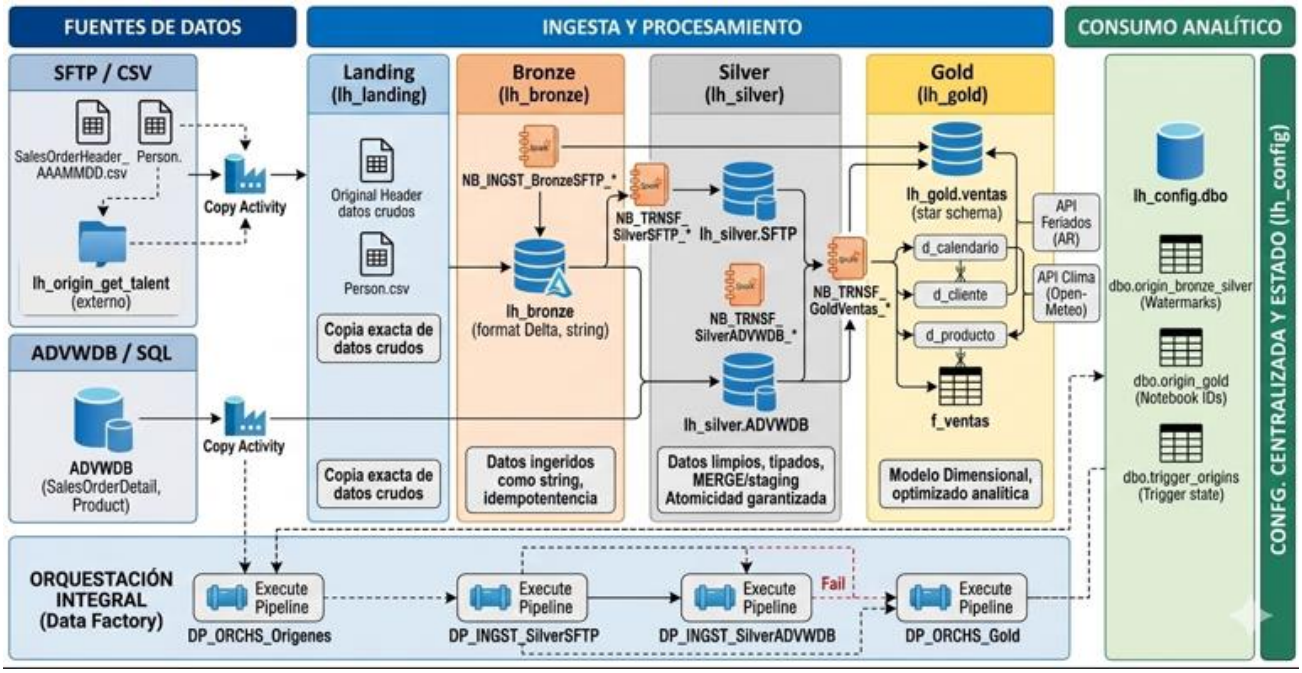
DP_ORCHS_Gold — Orquestador Gold

Lee la tabla lh_config.dbo.origin_gold ordenada por el campo orden y ejecuta cada notebook Gold en secuencia estricta mediante un ForEach. Entre notebooks se intercala una actividad Wait para

gestionar la capacidad Spark. El orden es mandatorio: DimCalendario debe existir antes de FactVentas, ya que ésta última hace JOIN sobre ella.

Flujo general de ejecución

El flujo completo de procesamiento del sistema puede representarse de la siguiente manera:



5. Modelo de Datos Gold

5.1 Modelo dimensional (Star Schema)

La capa Gold implementa un **modelo en estrella (star schema)** dentro del esquema lh_gold.ventas. Este modelo está optimizado para consultas analíticas de ventas: permite analizar el comportamiento de compras por período, cliente, producto y combinar esa información con el contexto de feriados y condiciones climáticas del día de la venta.

```
d_calendario                d_cliente
(PK: fecha)                 (PK: cliente_id)
        \                       /
         \                     /
          +----f_ventas-----+
         /  (PK: orden_id  \
        /    + detalle_id)  \
d_producto                  (fk: fecha, cliente_id, producto_id)
```

5.2 Definición de tablas Gold

Tabla	Esquema	Columnas clave	PK	Fuente
d_calendario	lh_gold.ventas	fecha, anio, mes, dia, es_feriado, clima_max_temp	fecha (DATE)	API Feriados AR + Open-Meteo
d_cliente	lh_gold.ventas	cliente_id, nombre, ap_paterno, ap_materno, persontype	cliente_id (INT)	Silver SFTP Person
d_producto	lh_gold.ventas	producto_id, nombre, numero, color, precio_lista, subcategoria, categoria	producto_id (INT)	Silver ADVWDB Product
f_ventas	lh_gold.ventas	orden_id, detalle_id, fecha_orden, cliente_id, producto_id, cantidad, precio_unit, total_linea	orden_id + detalle_id	Silver SFTP SOH + ADVWDB SOD

5.3 Criterios de filtrado y transformación

d_calendario

La dimensión calendario se construye generando programáticamente todos los días del rango de fechas relevante para las ventas. Para cada fecha se calculan atributos derivados (año, mes, día,

nombre del día, trimestre) y se enriquece con dos fuentes externas: la API del Gobierno Argentino devuelve los feriados nacionales (campo es_feriado), y la API Open-Meteo provee la temperatura máxima histórica (clima_max_temp). El rango de fechas es dinámico, determinado por el mínimo y máximo de OrderDate presente en los datos de ventas.

d_cliente

Se construye a partir de la tabla lh_silver.SFTP.Person, filtrando únicamente los registros con PersonType en ('SC', 'IN'), que corresponden a clientes individuales y tiendas según la taxonomía de AdventureWorks. Se normaliza el nombre completo y se conservan los identificadores necesarios para el JOIN con f_ventas a través del campo BusinessEntityID.

d_producto

Proviene de lh_silver.ADVWDB.Product. Se excluyen los productos discontinuados (FinishedGoodsFlag = 0 en versiones con ese flag disponible) y se enriquecen los campos con la jerarquía de categoría → subcategoría mediante JOINs contra las tablas auxiliares de AdventureWorks. El campo precio_lista (ListPrice) permite el cálculo de descuentos implícitos en f_ventas.

f_ventas — Tabla de hechos

La tabla de hechos integra SalesOrderHeader (SFTP, datos de la orden) con SalesOrderDetail (ADVWDB, líneas de detalle) a través del campo SalesOrderID. Cada fila representa una línea de venta individual. Las claves foráneas (fecha_orden → d_calendario, CustomerID → d_cliente, ProductID → d_producto) permiten análisis multidimensional. Las métricas almacenadas son: cantidad, precio unitario, descuento y total de línea.

6. Integraciones Externas (BONUS)

6.1 API Feriados Nacionales Argentina

Endpoint utilizado: **https://nolaborables.com.ar/api/v2/feriados/{anio}**. La API devuelve un JSON con la lista de feriados para el año solicitado. El notebook DimCalendario itera sobre los años del rango de fechas, llama a la API con requests.get(), parsea la respuesta y genera un conjunto de fechas feriadas. Cada fecha en la dimensión recibe el campo es_feriado (booleano) mediante un LEFT JOIN con este conjunto.

> *En caso de fallo de la API (timeout, error HTTP), el notebook continúa con is_feriado = False para todas las fechas, garantizando que un problema de conectividad externa no bloquee el pipeline completo.*

6.2 API Clima Open-Meteo

Endpoint utilizado: **https://archive-api.open-meteo.com/v1/archive**. Open-Meteo es un servicio gratuito que provee datos meteorológicos históricos sin necesidad de API key. Se consulta la temperatura máxima diaria (temperature_2m_max) para la ubicación de Buenos Aires (latitud -34.6, longitud -58.4) en el rango de fechas de la dimensión. La respuesta es un JSON con dos arrays paralelos (fechas y valores) que se transforman en un DataFrame de Spark para el JOIN final.

> *Esta integración es un componente BONUS que enriquece el análisis de ventas con contexto climático, permitiendo, por ejemplo, evaluar si las condiciones de temperatura influyen en la venta de ciertos productos estacionales.*

7. Manual de Ejecución

7.1 Prerrequisitos

Antes de ejecutar el pipeline por primera vez, se deben cumplir los siguientes requisitos en el workspace **gt-deuler-dev** de Microsoft Fabric:

- Los 5 lakehouses deben estar creados: lh_landing, lh_bronze, lh_silver, lh_gold, lh_config.
- Todos los notebooks deben estar importados en el workspace con sus IDs registrados en NB_CNFGS_SetupLhConfig.
- Los 4 pipelines JSON deben estar importados: DP_INGST_SilverSFTP, DP_INGST_SilverADVWDB, DP_ORCHS_Origenes, DP_ORCHS_Gold.
- El lakehouse externo lh_origin_get_talent (workspace WS-GT-DEMOS) debe contener los archivos CSV en las rutas configuradas.
- La capacidad de Fabric (SKU Trial o superior) debe estar activa.

7.2 Pasos de inicialización (primera vez)

#	Paso	Acción	Cómo
1	NB_CNFGS_SetupSchemas (una sola vez)	Crea todas las estructuras Delta necesarias en los 5 lakehouses	Workspace gt-deuler-dev Run All
2	NB_CNFGS_SetupLhConfig (una sola vez)	Carga tablas de config: trigger_origins, origin_bronze_silver, origin_gold	Workspace gt-deuler-dev Run All
3	DP_ORCHS_Origenes	Orquestador maestro: lanza SilverSFTP y SilverADVWDB secuencialmente	Run ahora o vía trigger diario

7.3 Ejecución incremental (operación normal)

Una vez completada la inicialización, la operación normal consiste en ejecutar únicamente **DP_ORCHS_Origenes**. Este pipeline detecta automáticamente el estado del watermark, procesa únicamente los datos nuevos y actualiza la configuración al finalizar. El flujo completo dura aproximadamente 20-30 minutos en el entorno trial.

> *DP_ORCHS_Origenes puede ser configurado con un trigger diario automático registrando el pipeline en lh_config.dbo.trigger_origins con diario_manana = 1. El sistema de triggers no está incluido en el alcance de este trabajo, pero la infraestructura de configuración está preparada para soportarlo.*

7.4 Reprocesamiento de datos

Para reprocesar un rango de datos ya procesado (por ejemplo, ante una corrección en los archivos fuente), se deben ejecutar los siguientes pasos en un notebook en Fabric:

```
-- 1. Resetear el watermark de la tabla a reprocesar
UPDATE lh_config.dbo.origin_bronze_silver
SET ultimo_valor_incremental = '20260221'
WHERE origen = 'SFTP' AND nombre_tabla = 'SalesOrderHeader';
```

```
-- 2. Opcionalmente, limpiar los datos existentes en Bronze y Silver
DROP TABLE IF EXISTS lh_bronze.SFTP.SalesOrderHeader;
DROP TABLE IF EXISTS lh_silver.SFTP.SalesOrderHeader;
```

Una vez ejecutados estos pasos, se vuelve a lanzar DP_ORCHS_Origenes. El sistema procesará nuevamente todos los archivos desde el watermark indicado.

8. Consideraciones Finales

8.1 Cumplimiento de requisitos del TP

El proyecto cumple con la totalidad de los requisitos establecidos en la consigna del trabajo práctico:

- Pipeline Landing → Bronze → Silver → Gold implementado y funcional para las dos fuentes de datos.
- Dos tablas de AdventureWorks: SalesOrderDetail y Product, acordadas con la cátedra.
- Archivos CSV sucios con convención de nombre tabla_AAAAMMDD.csv, con 2-4 archivos por tabla.
- Carga incremental (SalesOrderHeader, SalesOrderDetail) y total (Person, Product) implementadas.
- Lógica de reproceso disponible mediante reset de watermark en tabla de configuración.
- Parametrización completa: cero hardcodeos verificados en auditoría de 4 pipelines y 15 notebooks.
- Integración API Feriados Argentina en DimCalendario.
- BONUS: Integración API Clima Open-Meteo en DimCalendario.

8.2 Limitaciones conocidas del entorno trial

- El SKU Trial de Fabric limita la concurrencia de sesiones Spark, requiriendo los Waits entre notebooks secuenciales.
- La capacidad puede interrumpirse si el workspace está inactivo por más de 24 horas, requiriendo reactivación manual.
- El entorno de desarrollo usa LIMIT 1000 en las queries a ADVWDB para mantener tiempos de ejecución razonables. En producción, este límite debe eliminarse.

8.3 Posibles extensiones futuras

- Agregar un pipeline de calidad de datos entre Silver y Gold que valide rangos, nulos obligatorios y referencialidad entre dimensiones y hechos.
- Implementar el trigger automático diario utilizando la tabla trigger_origins ya disponible en lh_config.
- Conectar lh_gold a Power BI mediante Direct Lake para visualización en tiempo real del modelo dimensional.
- Agregar logging estructurado en cada notebook, escribiendo métricas de ejecución (filas procesadas, duración, estado) en una tabla lh_config.dbo.execution_log.

— Fin del Documento Técnico —

TP Final · Ingeniería de Datos en Azure / Microsoft Fabric · 2026